INTRODUCCIÓ ALS ORDINADORS

INFORME PRÀCTICA 7

Grau d'Enginyeria Informàtica

Universitat de Barcelona

Alumne: Cristian Prado
Grup de Pràctiques: F
Curs: 2022/2023
Semestre de primavera

INTRODUCCIÓ

En aquesta pràctica es fa un aprofundiment en i8085, utilitzant la pantalla de text i el teclat per poder fer programes. En particular, a partir de l'ús de la pila, dels registres que ofereix i8085 i de la gestió de la memòria, es faran programes que pugin fer operacions amb nombres de 3 xifres.

OBJECTIUS

Com a objectius de la pràctica es vol adquirir una capacitat per saber com s'està gestionant la memòria en cada moment de l'execució del programa i programar en llenguatge assemblador, mitjançant i8085, diversos programes capaços de fer les operacions que es demanen, aplicant els coneixements de les classes de teoria.

IMPORTANT

Cal esmentar que el codi que es presentarà a continuació és el codi sencer. En aquest cas, no s'han fet 3 codis diferents, ja que s'ha considerat que en fer-lo junt, les modificacions que hi havia en cada apartat es feien més fàcils. D'aquesta forma, moltes accions que es feien en el codi feien que en aspectes com la resta i la suma fossin un pur tràmit. Així doncs, es contestaran les preguntes de cada apartat, però tenint en compte que només es presenta un únic codi que fa totes les operacions demanades.

1. Suma de dos valors introduïts per consola

Dissenyeu una subrutina que a patir de dos nombres introduïts pel teclat del simulador i8085 faci la suma i presenti el resultat en la pantalla de text del i8085. Feu servir els adreçaments directe i indirecte i indiqueu al codi on tenim aquests adreçaments. Recordeu que els valors introduïts per teclat es donen en codificació ASCII. Que passa si el valor resultant és superior a 127? En el cas del 8085 penseu en un codi que sumi unitats amb unitats, desenes amb desenes i centenes amb centenes per solucionar aquest problema.

En el cas de que el valor sigui superior a 127, com estem treballant amb registres de 8 bits, aleshores si es guarda un valor superior a aquest es produeix un overflow i ens dona un nombre negatiu.

Tasca 1.1 Pugeu el codi. Com gestioneu el problema del signe? Com gestioneu el problema del overflow?

El problema del signe en veritat no és cap problema, ja que les xifres dels nombres es tracten de forma individual i d'aquesta forma també s'evita el overflow en els nombres per separat. Ara bé, si en el moment de la suma el resultat és superior a 999 en decimal, aleshores es produeix en el programa un overflow perquè estem treballant amb 3 xifres a cada nombre. Per tant, el que es fa es guardar en una etiqueta anomenada carry el possible carry que puguem tenir en la suma de les centenes. En cas que es produeixi, aleshores en el moment de mostrar el resultat, es mostrarà per pantalla tres X's, tal que XXX.

Tasca 1.2 Considereu la suma en Ripes. Quan podeu tenir problemes d'overflow o carry? Considereu que podeu treballar amb 32 però també amb 64 bits. On estaria el límit en aquest segon cas?

De la mateixa forma que en i8085, a Ripes també es pot produir un overflow o carry. No obstant, si estem traballant amb 32 bits, aleshores es produeix quan el nombre és més gran que 2^31 -1 i en el cas de 64 bits, quan aquest sigui més gran que 2^63 -1.

A continuació es presenten les subrutines característiques d'aquesta part, que són les sumes per unitats, desenes i centenes:

```
;Subrutina per fer la suma de les unitats

.org 550h
unitats_suma:
push h
lxi h, unitat1
mov a, M
lxi h, unitat2
add M
```

```
cpi 0Fh
jnc modifica_unitat2
cpi Ah
jnc modifica_unitat
lxi h, unitatR
mov M, a
jmp final
modifica_unitat:
sui Ah
lxi h, unitatR
mov M,a
lxi h, desenaR
mvi M, 1
jmp final
modifica_unitat2:
sui 10h
adi 6
lxi h, unitatR
mov M,a
lxi h, desenaR
mvi M, 1
final:
pop h
ret

;Subrutina per fer la suma de les desenes

.org 600h
desenes_suma:
push h
lxi h, desenaR
mov a, M
lxi h, desena1
add M
lxi h, desena2
add M
cpi 0Fh
jnc modifica_desena2
cpi Ah
jnc modifica_desena
lxi h, desenaR
```

```
mov M, a
jmp final1
modifica_desena:
sui Ah
lxi h, desenaR
mov M,a
lxi h, centenaR
mvi M, 1
jmp final1
modifica_desena2:
sui 10h
adi 6
lxi h, desenaR
mov M,a
lxi h, centenaR
mvi M, 1
final1:
pop h
ret

;Subrutina per fer la suma de les centenes

.org 700h
centenes_suma:
push h
lxi h, centenaR
mov a, M
lxi h, centena1
add M
lxi h, centena2
add M
cpi 0Fh
jnc modifica_centena2
cpi Ah
jnc modifica_centena
lxi h, centenaR
mov M, a
jmp final2
modifica_centena:
sui Ah
lxi h, centenaR
```

```
mov M,a
lxi h, carry
mvi M, 1
jmp final2
modifica_centena2:
sui 10h
adi 6
lxi h, centenaR
mov M,a
lxi h, carry
mvi M, 1
final2:
pop h
ret
```

2. Resta de dos valors introduïts per consola

Dissenyeu una subrutina que a patir de dos nombres introduïts pel teclat del simulador i8085 faci la resta i presenti el resultat en la pantalla de text del i8085. Feu servir els adreçaments directe i indirecte i indiqueu al codi on tenim aquests adreçaments.

Tasca 2. Pugeu el codi de la resta. Com gestioneu el problema del signe? Com gestioneu el problema del carry?

En aquest cas, veiem que la resta se'ns pot presentar de dues formes: nombre1 - nombre2 i -nombre1 + nombre2. En el primer cas, si el primer nombre és més gran que el segon en valor absolut, no hi ha cap problema. En aquest cas, si en la resta xifra per xifra es produeix un carry (és a dir 8-9 ->18-9 =9 i m'emporto 1), el que es fa és primer de tot sumar-li Ah a l'acumulador, de forma que després podrem guardar el 9 en l'etiqueta corresponent. Després, s'agafa l'etiqueta de la xifra del segon nombre i se li suma 1. De forma que podem seguir operant en el següent pas, és a dir la xifra següent. Per altra banda, si ens trobem que en el primer cas el primer nombre és més petit que el segon en valor absolut, aleshores el que es fa és fer un canvi de nombres. És a dir, el que es fa és canviar la xifra d'unitats del primer nombre per la del segon, després amb les desenes i finalment amb les centenes. D'aquesta forma, ara el primer

nombre és el segon original i el primer original passa a ser el segon. Exemple: 89-117 -> Nombre 1 = 89, Nombre 2 = 117 -> Nombre 1 = 117, Nombre 2 = 89 -> 117-89.
D'aquesta forma, podem fer la resta sense cap problema, com abans. No obstant això, el que es fa és guardar el signe del nombre en valor absolut més gran. Posteriorment, quan es mostra el resultat a pantalla, també es mostra el signe del nombre més gran. Al final el que estem fent, tenint en compte l'exemple, és -(117-89).
En el segon cas, veiem que és el mateix cas, ja que sempre es tindrà en compte el signe del nombre més gran en valor absolut, amb la qual cosa si el primer nombre és més gran que el segon, tot i que sigui -nombre1 +nombre2, es guarda el signe del nombre 1 i després es posa al resultat. I si el segon nombre és més gran que el primer, es tornen a fer les mateixes passes que abans, guardant el signe corresponent.
A continuació es mostren les subrutines per fer les operacions de la resta de les xifres, és a dir, unitats, desenes i centenes:

```
;Subrutina per fer la resta de les unitats

.org 750
unitats_resta:
push h
lxi h, unitat1
mov a, M
lxi h, unitat2
sub M
jm modifica_uResta
lxi h, unitatR
mov M, a
jmp end
modifica_uResta:
adi Ah
lxi h, unitatR
mov M, a
lxi h, desena2
mov a, M
adi 1
mov M, a
end:
pop h
ret
```

```
;Subrutina per fer la resta de les desenes

.org 800h
desenes_resta:
push h
lxi h, desena1
mov a, M
lxi h, desena2
sub M
jm modifica_dResta
lxi h, desenaR
mov M, a
jmp end1
modifica_dResta:
adi Ah
lxi h, desenaR
mov M, a
lxi h, centena2
mov a, M
adi 1
mov M, a
end1:
pop h
ret

;Subrutina per fer la suma de les centenes

.org 850h
centenes_resta:
push h
lxi h, centena1
mov a, M
lxi h, centena2
sub M
jm modifica_cResta
lxi h, centenaR
mov M, a
jmp end2
modifica_cResta:
adi Ah
```

```
lxi h, centenaR
mov M, a
lxi h, carry ; Important per desprÈs detectar el overflow
mov a, M
adi 1
mov M, a
end2:
pop h
ret
```

3. Ensamblant el codi

A partir dels codis generats en els apartats 1 i 2, feu un programa capaç de fer sumes, restes, AND's i OR's

Tasca 3. Pugeu el codi final.

A continuació, es dona el codi del programa que fa les operacions demanades. Cal esmentar que les explicacions del codi estan donades en el mateix codi. Tot i així, la idea del codi ha estat tractar per separat les diferents xifres dels nombres, i mostrar-les per pantalla a mesura que s'anaven registrant a memòria. Per altra banda, ha estat important gestionar els signes, ja que en el cas de la resta es feia difícil tractar els casos on la resta natural (nombre1-nombre2 on nombre1 és més gran que nombre2 en valor absolut) no es donava, i s'havien d'intercanviar els papers de nombre1 i nombre2 per tal de poder fer l'operació correctament, mostrant després el resultat correcte per pantalla. Per altra banda, una vegada es mostrava el següent format en la pantalla de text: nombre 1 (operació) nombre2 =; aleshores es començava a fer les operacions al programa, en la que ja es disposava de les xifres guardades a memòria. Per tant, després de molts steps, s'arribava a mostrar per pantalla el resultat de l'operació corresponent, on es mostrava amb el signe negatiu en cas de que el necessités o sense signe en el cas que el resultat fos positiu. Dit això, el codi és el següent:

```
;################################
; MEMORIA DE DADES
;################################
```

```
.data 50h
unitat1: db 0  ; unitat que enviara a pantalla
desena1: db 0  ; desena que enviara a pantalla
centena1: db 0 ; centena que enviara a pantalla
unitat2: db 0  ; unitat que enviara a pantalla
desena2: db 0  ; desena que enviara a pantalla
centena2: db 0 ; centena que enviara a pantalla
unitatR: db 0  ; unitat que enviara a pantalla
desenaR: db 0  ; desena que enviara a pantalla
centenaR: db 0 ; centena que enviara a pantalla
carry: db 0   ; mira si hi ha carry
signe1: db 0h   ; guarda el signe del primer valor
signe2: db 0h   ; guarda el signe del segon valor
signeresultat: db 0h ; guarda el signe del resultat
memoria_pantalla: dw e000h ; ens servira per avanÁar en la pantalla de
text
;################################
; MEMORIA DE PROGRAMA
;################################
.org 100h
;#######################################################################
########################################
; Cal esmentar en aquest codi que els valors van apareixent en
pantalla a mesura que es van introduÔnt, com si
; fos una calculadora.
;#######################################################################
########################################



;##########################################
; INTRODUIM EL PRIMER VALOR (SIGNE O XIFRA)
;##########################################

;#######################################################################
#########################################################
; S'ha d'observar que cada vegada que s'introdueix un valor, aquest
s'ha d'actualitzar amb les altres xifres del nombre
; de forma que les unitats passen a ser desenes i les desenes a
centenes.
```

```
;#################################################################
#######################################################
lhld memoria_pantalla
mov d,h ; Utilitzarem d i e per anar guardant la direcciÛ de memÚria
de la pantalla, de forma que podem anar treballant de mentres amb el
parell HL
mov e,l
in 0h
cpi 30h     ; Volem veure si s'ha introduÔt un signe negatiu, positiu,
and (&) o or (|). AixÚ es far‡ en cada xifra del nombre
jnc continuaOpera ; Es far‡ el salt si el que s'ha introduÔt Ès un
nombre
lxi h, signe1
mov M,a
mov h,d
mov l,e
mov M, a
inx h
mov d,h
mov e,l
in 0h
continuaOpera:
sui 30h
lxi h, unitat1
mov M,a
mov b,M
mov h,d
mov l,e
call mostra_digit
inx h
mov d,h
mov e,l
;##################################
; INTRODUIM LA SEGONA XIFRA O SIGNE
;##################################
in 0h
cpi 30h
jc segon_nombre
cpi 40h ; A partir d'aquest valor podem comparar per veure si s'ha
introduÔt un nombre o no, ens interessa veure si | (7Ch) s'ha entrat
per teclat
```

```
jnc segon_nombre
lxi h, desena1
mov M,b
lxi h, unitat1
sui 30h
mov M,a
mov b,M
lxi h, desena1
mov c,M
mov h,d
mov l,e
call mostra_digit
inx h
mov d,h
mov e,l
;####################################
; INTRODUIM LA TERCERA XIFRA O SIGNE
;####################################
in 0h
cpi 30h
jc segon_nombre
cpi 40h ; A partir d'aquest valor podem comparar per veure si s'ha
introduÔt un nombre o no, ens interessa veure si | (7Ch) s'ha entrat
per teclat
jnc segon_nombre
lxi h, centena1
mov M,c
lxi h, desena1
mov M,b
lxi h, unitat1
sui 30h
mov M,a
mov h,d
mov l,e
call mostra_digit
inx h
mov d,h
mov e,l

;#####################################################
; INTRODUIM EL SEGON VALOR I L'OPERACIO QUE VOLEM FER
```

```
;#################################################

in 0h
segon_nombre:
lxi h, signe2
mov M, a
mov h,d
mov l,e
mov M,a
inx h
mov d,h
mov e,l

lxi h, unitat2
in 0h
cpi 3Ah
jnc finalOpera
sui 30h
mov M,a
mov b,M
mov h,d
mov l,e
call mostra_digit
inx h
mov d,h
mov e,l
in 0h
cpi 3Ah
jnc finalOpera
lxi h, desena2
mov M,b
lxi h, unitat2
sui 30h
mov M,a
mov b,M
lxi h, desena2
mov c,M
mov h,d
mov l,e
call mostra_digit
inx h
```

```
mov d,h
mov e,l
in 0h
cpi 3Ah

; Volem veure que si el que s'ha introduÔt per pantalla Ès un igual
(=) aleshores fem un salt com finalitzaciÛ de la introducciÛ de
nombres.
;AixÚ es far‡ en cada introducciÛ de xifra

jnc finalOpera
lxi h, centena2
mov M,c
lxi h, desena2
mov M,b
lxi h, unitat2
sui 30h
mov M,a
mov h,d
mov l,e
call mostra_digit
inx h
mvi a, 3dh
mov M, a
inx h
mov d,h
mov e,l

jmp comparacions ; Es fa un salt a la part de comparacions de nombres
i elecciÛ d'operaciÛ.

finalOpera:
mov h,d
mov l,e
mov M, a
inx h
mov d,h
mov e,l

comparacions:
```

```
;#################################################################
#########################################################
; COMPARACI” ENTRE ELS DOS NOMBRES I SIGNES
; AquÌ es far‡ la comparaciÛ dels signes per veure quin cas Ès l'idoni
per operar. Es consideren els casos +/+, +/-, -/+ i -/-.
; A mÈs a mÈs, tambÈ es fan les portes OR i AND
;#################################################################
#########################################################
lxi h, signe1
mov a, M
sui 2dh
jz restes ; Es comprova si el primer nombre tÈ signe negatiu. En cas
positiu, aleshores es fa un salt a restes
      lxi h, signe2
      mov a,M
      sui 2bh ; Es comprova si el segon nombre tÈ signe positiu (suma)
      jz suma
      adi 2bh
      sui 2dh ; Es comprova si el segon nombre tÈ signe negatiu
(resta)
      jz diferents
      adi 2dh
      sui 26h ; Es comprova si el segon nombre tÈ signe AND (&)
      jz portaand
      adi 26h
      sui 7ch ; Es comprova si el segon nombre tÈ signe OR (|)
      jz portaor
restes:
      lxi h, signe2
      mov a,M
      sui 2dh
      jnz diferents
      lxi h, signe1
      mov a, M
      lxi h, signeresultat ; Si els dos nombres tenen signe negatiu,
aleshores es guarda el signe en signeresultat i es fa una suma
      mov M, a
suma:
      call unitats_suma
      call desenes_suma
      call centenes_suma
```

```
      lxi h, carry
      mov a,M
      sui 1h
      jz overflow ;En el cas que hi hagi un valor a carry, aleshores
vol dir que tenim overflow, i es fa un salt
      jmp mostrarResultats

;#######################################################################
#############################################################
; COMPARACI" ENTRE ELS DOS NOMBRES
; AquÌ es far‡ la comparaciÛ dels nombre per saber quin Ès mÈs gran, i
per tant quin dels dos far‡ que predomini el signe
; en l'operaciÛ. En particular, el procediment que es seguir‡ Ès moure
a acumulador el contingut de la xifra del primer nombre
; i restar-li la xifra corresponent del segon, en ordre decreixent
(centenes, desenes i unitats). Si en algun cas es dona que
; el valor de l'acumulador Ès negatiu, aleshores el segon nombre en
valor absolut Ès mÈs gran que el primer, i per tant predominar‡
; el signe del segon nombre.
;#######################################################################
#############################################################


diferents:
      lxi h, centena1
      mov a, M
      lxi h, centena2
      sub M
      jm canvi
      mov a, 0h
      lxi h, desena1
      mov a, M
      lxi h, desena2
      sub M
      jm canvi
      mov a, 0h
      lxi h, unitat1
      mov a, M
      lxi h, unitat2
      sub M
      jm canvi
```

```
      lxi h, signe1
      mov a, 0h
      mov a,M
      lxi h, signeresultat
      mov M, a
      jmp resta

;#######################################################################
##############################################################
; CANVI DE NOMBRES
; AquÌ es fa un canvi de forma que el primer nombre passa a ser el
segon i el segon passa a ser el primer. D'aquesta forma no es
; tindr‡ problemes en fer la resta. El procediment Ès utilitzar els
registres B i H juntament amb l'acumulador, i anant actualitzant
; els dos nombres, intercanviant les xifres.
;#######################################################################
##############################################################

canvi:
      mov b, 0h
      lxi h, centena1
      mov a, M
      mov b,a
      lxi h, centena2
      mov a, M
      lxi h, centena1
      mov M, a
      lxi h, centena2
      mov M, b
      mov b, 0h
      lxi h, desena1
      mov a, M
      mov b,a
      lxi h, desena2
      mov a, M
      lxi h, desena1
      mov M, a
      lxi h, desena2
      mov M, b
      mov b, 0h
      lxi h, unitat1
```

```
      mov a, M
      mov b,a
      lxi h, unitat2
      mov a, M
      lxi h, unitat1
      mov M, a
      lxi h, unitat2
      mov M, b
      lxi h, signe2
      mov a, 0h
      mov a,M
      lxi h, signeresultat
      mov M, a
resta:
      call unitats_resta
      call desenes_resta
      call centenes_resta
      jmp mostrarResultats
portaand:
      call and
      jmp mostrarResultats
portaor:
      call or

; Finalment, aqui es mostren els resultats de l'operaciÛ

mostrarResultats:
lxi h, signeresultat
mov a, M
sui 2dh
jz posar_signe
jmp continuaMostrant
posar_signe:
mov h,d
mov l,e
adi 2dh
mov M, a
inx h
mov d,h
mov e,l
continuaMostrant:
```

```
lxi h, centenaR
mov a, M
mov h,d
mov l,e
call mostra_digit
inx h
mov d,h
mov e,l
lxi h, desenaR
mov a, M
mov h,d
mov l,e
call mostra_digit
inx h
mov d,h
mov e,l
lxi h, unitatR
mov a, M
mov h,d
mov l,e
call mostra_digit
inx h
jmp finalPrograma

; En el cas que es produeixi el overflow, aleshores es posaran 3 X's
de forma que el resultat ser‡ XXX
overflow:
mov h,d
mov l,e
adi 58h
mov M,a
inx h
mov M,a
inx h
mov M,a
finalPrograma:
hlt

;################################
;SUBRUTINES UTILITZADES
;################################
```

```
;Subrutina per mostrar els dÌgits dels nombres

.org 500h
mostra_digit:
push h
;primer valor en pantalla
adi 30h
mov M,a
pop h
ret

;Subrutina per fer la suma de les unitats

.org 550h
unitats_suma:
push h
lxi h, unitat1
mov a, M
lxi h, unitat2
add M
cpi 0Fh
jnc modifica_unitat2
cpi Ah
jnc modifica_unitat
lxi h, unitatR
mov M, a
jmp final
modifica_unitat:
sui Ah
lxi h, unitatR
mov M,a
lxi h, desenaR
mvi M, 1
jmp final
modifica_unitat2:
sui 10h
adi 6
lxi h, unitatR
mov M,a
lxi h, desenaR
```

```
mvi M, 1
final:
pop h
ret

;Subrutina per fer la suma de les desenes

.org 600h
desenes_suma:
push h
lxi h, desenaR
mov a, M
lxi h, desena1
add M
lxi h, desena2
add M
cpi 0Fh
jnc modifica_desena2
cpi Ah
jnc modifica_desena
lxi h, desenaR
mov M, a
jmp final1
modifica_desena:
sui Ah
lxi h, desenaR
mov M,a
lxi h, centenaR
mvi M, 1
jmp final1
modifica_desena2:
sui 10h
adi 6
lxi h, desenaR
mov M,a
lxi h, centenaR
mvi M, 1
final1:
pop h
ret
```

```
;Subrutina per fer la suma de les centenes

.org 700h
centenes_suma:
push h
lxi h, centenaR
mov a, M
lxi h, centena1
add M
lxi h, centena2
add M
cpi 0Fh
jnc modifica_centena2
cpi Ah
jnc modifica_centena
lxi h, centenaR
mov M, a
jmp final2
modifica_centena:
sui Ah
lxi h, centenaR
mov M,a
lxi h, carry
mvi M, 1
jmp final2
modifica_centena2:
sui 10h
adi 6
lxi h, centenaR
mov M,a
lxi h, carry
mvi M, 1
final2:
pop h
ret

;Subrutina per fer la resta de les unitats

.org 750
unitats_resta:
push h
```

```
lxi h, unitat1
mov a, M
lxi h, unitat2
sub M
jm modifica_uResta
lxi h, unitatR
mov M, a
jmp end
modifica_uResta:
adi Ah
lxi h, unitatR
mov M, a
lxi h, desena2
mov a, M
adi 1
mov M, a
end:
pop h
ret

;Subrutina per fer la resta de les desenes

.org 800h
desenes_resta:
push h
lxi h, desena1
mov a, M
lxi h, desena2
sub M
jm modifica_dResta
lxi h, desenaR
mov M, a
jmp end1
modifica_dResta:
adi Ah
lxi h, desenaR
mov M, a
lxi h, centena2
mov a, M
adi 1
mov M, a
```

```
end1:
pop h
ret

;Subrutina per fer la suma de les centenes

.org 850h
centenes_resta:
push h
lxi h, centena1
mov a, M
lxi h, centena2
sub M
jm modifica_cResta
lxi h, centenaR
mov M, a
jmp end2
modifica_cResta:
adi Ah
lxi h, centenaR
mov M, a
lxi h, carry ; Important per desprÈs detectar el overflow
mov a, M
adi 1
mov M, a
end2:
pop h
ret

;#################################################################
;     OPERACIONS L"GIQUES
;#################################################################
.org 1000h

;#################################################################
;           DECODIFICAR DECIMAL - BINARI
;#################################################################
dec_to_bin:
      mvi B, 0h
      mvi H, 0h
```

```
      ; En primer lloc gestionem les unitats. Simplement sumem el
contingut de les unitats al registre L
      lda unitat1
      mov L, A


      ; Ara es faran les desenes, Ah tants cops com digui el valor de
l'acumulador
      lda desena1
      CPI 00h
      jz hex_centenes_valor1
      mvi C, 0Ah ; 0Ah Ès  10 en decimal

      loop_desenes_valor1:
      dad B
      dcr A
      jnz loop_desenes_valor1

      ; De la mateixa forma amb les centenes
      hex_centenes_valor1:
      lda centena1
      CPI 00h
      JZ hex_end_valor1
      mvi C, 64h ; 64h Ès 100 en decimal
      loop_centenes_valor1:
      dad B
      dcr A
      jnz loop_centenes_valor1

      hex_end_valor1:
      ; Finalment guardem el contingut de L a E i el de H a D (parell
DE)
    MOV E, L
    MOV D, H

      ; Repetim el mateix procËs, ara guardant el resultat al parell
BC
      ; Cal reiniciar les xifres del parell HL
      MVI H, 0h
      MVI L, 0h
      MVI B, 0h
```

```
      MVI C, 0h
    ; Xifra 2
    lda unitat2
    mov L, A

    lda desena2
      CPI 00h
      JZ hex_centenes_valor2
    mvi C, 0Ah
    loop_desenes_valor2:
    dad B
    dcr A
    jnz loop_desenes_valor2

      hex_centenes_valor2:
    lda centena2
      CPI 00h
      JZ hex_end_valor2
    mvi C, 64h

    loop_centenes_valor2:
    dad B
    dcr A
    jnz loop_centenes_valor2

      hex_end_valor2:
    MOV C, L
    MOV B, H
      ret

;#################################################################
;           DECODIFICAR BINARI - DECIMAL
;#################################################################
bin_to_dec:

      MVI B, FFh
      MVI C, 9Ch; FF9Ch es -100 en Ca2
      MVI D, 00h; El nombre de centenes
      MVI E, 00h, El nombre de desenes
```

```
loop1:
      DAD B; Restem 100 a HL mentre es pugui
      JNC loop1_1
      INR D
      JMP loop1

loop1_1:
      MVI B, 00h
      MVI C, 64h
      DAD B

MVI B, FFh;
MVI C, F6h; Aixo es 10 en Ca2
loop2:
      DAD B ; Restem 10 a HL mentre es pugui
      JNC loop2_2
      INR E
      JMP loop2
loop2_2:

      MVI B, 00h
      MVI C, 0Ah
      DAD B



; Finalment guardem les unitats
MOV A, L
LXI H, unitatR
MOV M, A

LXI H, desenaR
MOV M, E

LXI H, centenaR
MOV M, D
ret

;#################################################################
;           OPERACI" AND
;#################################################################
```

```
; Ara que tenim els nombres en binari, es poden fer les portes que es
vulgui
and:
      PUSH H
      PUSH B
      PUSH D
      call dec_to_bin
      MOV A, B
      ANA D
      MOV H, A
      MOV A, C
      ANA E
      MOV L, A ; El resultat apareix al parell de registres HL
      call bin_to_dec

      POP D
      POP B
      POP H
      ret

;################################################################
;           OPERACI" OR
;################################################################
or:
            PUSH H
      PUSH B
      PUSH D
      call dec_to_bin
      MOV A, B
      ORA D
      MOV H, A
      MOV A, C
      ORA E
      MOV L, A ; El resultat apareix al parell de registres HL
      call bin_to_dec

      POP D
      POP B
      POP H
      ret
```

Questió 1: Quina diferència hi ha entre la suma i la OR?

i) són iguals

ii) la OR és una operació lògica i la suma és una operació aritmètica.

iii) La OR és una operació aritmètica i la suma és una operació lògica

iv) cap de les anteriors és correcta.

La resposta correcta és la ii), la OR és una operació lògica i la suma és una operació aritmètica.

Questió 2: La instrucció STA 1234h

i) és una operació que carrega el contingut de la posició de memòria 1234h en l'acumulador

ii) fa servir adreçament directe

iii) fa servir adreçament inmediat

iv) totes són certes

La resposta correcta és la ii), fa servir un adreçament directe, s'està dient la direcció en específic on es vol guardar el contingut de l'acumulador.

CONCLUSIONS

Com a conclusions d'aquesta pràctica tenim que s'ha pogut aprendre a fer programes en assembly mitjançant i8085. S'ha après que en cada moment es pot comprovar com s'està comportant el programa i a partir d'això solucionar possibles errors de programació, ja sigui per la gestió de la memòria o dels registres. A més a més, s'ha pogut aprendre a poder mostrar per pantalla de text els nombres o caràcters que es demanen per teclat. Per tant, s'ha pogut fer, a partir de tot l'esmentat, sumes, restes, AND i OR en el programa presentat, tractant el carry i l'overflow.